

A practical, code-first reference covering the core and modern JavaScript concepts (ES6+) that power real-world MERN stack (MongoDB, Express, React, Node.js) development — with definitions, real-life analogies, and fully explained code examples.

Table of Contents

1.	Values & Variables
2.	Data Types
3.	Expressions & Operators
4.	Control Statements & Loops
5.	Modern JS (ES6+): let/const, Arrow Functions, Template Literals
6.	Destructuring, Spread & Rest Operators
7.	ES Modules (import/export)
8.	Arrays & Array Methods
9.	Strings & String Methods
10.	Date and Time
11.	Math Object
12.	DOM, BOM & Window Object
13.	Events in JavaScript
14.	Timing Events: setTimeout & setInterval
15.	Scope, Hoisting & Closures
16.	Asynchronous JS: Callbacks, Promises, Async/Await
17.	The Event Loop
18.	Prototypes, this, call, apply & bind

1 Values & Variables

Definition: A variable is a named container used to store a value in memory so it can be reused or changed later. JavaScript gives us three ways to declare one: `var`, `let`, and `const`.

Real-life example: Think of a variable like a labeled storage box. A box labeled "age" can hold a value like 25. A `const` box is sealed shut once packed (like a birth certificate — it never changes), while a `let` box can be opened and re-packed anytime (like a wallet — the amount of cash inside keeps changing).

MERN relevance: Every Express route handler, React component, and Mongoose schema relies on clean variable declarations. Using `const` by default prevents accidental reassignment of req/res objects, state, or DB connections.

Example 1 — var vs let vs const

```
var a = 10; // function-scoped, avoid in modern code
let b = 20; // block-scoped, reassignable
const c = 30; // block-scoped, cannot be reassigned

b = 25; // ✔ works
c = 35; // ✖ TypeError: Assignment to constant variable
```

Explanation: Line 1 declares `a` with `var` (old-style, function-scoped). Line 2 declares `b` with `let`, which allows the value to be changed later — and indeed, line 5 successfully updates it to 25. Line 3 declares `c` as a constant; attempting to reassign it on line 6 throws an error because `const` variables can never be given a new value.

Example 2 — Block scope demonstration

```
{
  let x = "inside block";
  console.log(x); // "inside block"
}
console.log(typeof x); // "undefined" — x doesn't exist here
```

Explanation: The curly braces `{ }` create a block. `x` is declared with `let` inside that block, so it only exists there — printing it inside works fine. Once we step outside the block and try `typeof x`, JavaScript reports "undefined" because `x` was never created in the outer scope.

Example 3 — const with objects (mutation is allowed)

```
const user = { name: "Ali", age: 22 };
user.age = 23; // ✔ allowed, we're mutating not reassigning
user = {}; // ✖ TypeError
```

Explanation: `const` only locks the variable name from being pointed at a new value — it does not freeze the contents of an object. So changing `user.age` (line 2) is fine because we are editing what is inside the box, not replacing the box itself. Trying to assign a brand-new object to `user` (line 3) fails because that would replace the box.

Example 4 — Multiple declarations

```
let firstName = "Sara", lastName = "Khan", age = 28;
console.log(`${firstName} ${lastName} is ${age}`);
```

Explanation: A single `let` statement can declare several variables at once, separated by commas. This shortens code when initializing related values together (like fields for one person), and the template literal on line 2 then combines them into one readable string.

Example 5 — Naming rules & camelCase convention

```
const userFirstName = "Ahmed"; // ✔ camelCase (standard in JS)
const _privateVar = 1; // ✔ underscore allowed
const $element = "div"; // ✔ $ allowed (common in DOM/jQuery style)
// const 2ndValue = 5; // ✖ cannot start with a number
```

Explanation: Variable names can contain letters, digits, underscores, and dollar signs, but they can never start with a digit. The convention in JavaScript is camelCase (`userFirstName`), which keeps multi-word names readable without spaces.

2 Data Types

Definition: A data type describes what kind of value a variable holds and what operations can be performed on it. JavaScript has 7 primitive types (`String` , `Number` , `Boolean` , `undefined` , `null` , `BigInt` , `Symbol`) plus one reference type: `Object` (which includes arrays and functions).

Real-life example: Imagine sorting mail into different trays: letters (strings), invoices with amounts (numbers), yes/no forms (booleans), and empty envelopes (null/undefined). You handle each tray differently, just like JavaScript treats each data type differently.

Example 1 — Primitive types

```
const name    = "Zainab"; // String
const age     = 24;       // Number
const isAdmin = true;     // Boolean
let notSet    ;          // undefined
const empty   = null;     // null (intentional "no value")
const big     = 123456789012345678901234567890n; // BigInt
```

Explanation: Each line stores a different primitive type: text in quotes is a String, a plain number is a Number, `true` / `false` is a Boolean, a declared-but-unassigned variable is `undefined` , `null` is used when we deliberately want "no value", and appending `n` to a huge number creates a BigInt for numbers too large for the normal Number type.

Example 2 — typeof operator

```
console.log(typeof "hello"); // "string"
console.log(typeof 42);      // "number"
console.log(typeof undefined); // "undefined"
console.log(typeof null);    // "object" (famous JS quirk/bug)
console.log(typeof []);      // "object"
console.log(typeof {});      // "object"
```

Explanation: `typeof` returns a string naming the type of a value. Most results are intuitive, but `typeof null` returns `"object"` — a long-standing bug in JavaScript kept for backward compatibility. Arrays also report as `"object"` since arrays are technically a special kind of object.

Example 3 — Reference type (Object) — used everywhere in MERN

```
const product = {
  id: 101,
  title: "Wireless Mouse",
  price: 19.99,
  inStock: true
};
// This is the exact shape of data returned from a MongoDB document via Mongoose
```

Explanation: An object groups related values under named keys (id, title, price, inStock). This mirrors exactly how a MongoDB document looks once Mongoose converts it into a JavaScript object for your Express or React code to use.

Example 4 — Type coercion (implicit conversion)

```
console.log("5" + 3); // "53" (number converted to string)
console.log("5" - 3); // 2 (string converted to number)
console.log(1 == "1"); // true (loose equality coerces types)
console.log(1 === "1"); // false (strict equality - always prefer this)
```

Explanation: The `+` operator with a string and a number converts the number to a string and joins them (line 1), while `-` forces both sides into numbers (line 2). `==` compares values after converting types, so `1 == "1"` is true, but `===` checks both value and type without converting, so it correctly reports false.

Example 5 — Explicit type conversion

```
const raw = "42";
console.log(Number(raw)); // 42
console.log(String(42)); // "42"
console.log(Boolean(0)); // false
console.log(parseInt("20px")); // 20 - common when reading form/query values
```

Explanation: Instead of relying on automatic coercion, we can convert types on purpose: `Number()` turns text into a number, `String()` turns a number into text, `Boolean()` converts a value to true/false, and `parseInt()` reads leading digits out of a string (useful for values like "20px" coming from form inputs) while ignoring the rest.

3 Expressions & Operators

Definition: An operator performs an action on one or more values (operands) to produce a result — the combination of operator and operands is called an expression. Examples include arithmetic (`+` , `-`), comparison (`===`), logical (`&&`), and newer operators like nullish coalescing (`??`) and optional chaining (`?.`).

Real-life example: Think of a calculator: pressing "5 + 3" is an expression that produces 8. Logical operators are like everyday decisions — "if it's raining AND I have an umbrella, I'll go outside" mirrors `if (isRaining && hasUmbrella)` .

Example 1 — Arithmetic & assignment operators

```
let total = 100;
total += 50; // 150
total -= 20; // 130
total *= 2; // 260
total **= 2; // 67600 (exponent)
console.log(10 % 3); // 1 (modulus — used for pagination logic)
```

Explanation: `+=` , `-=` , `*=` , and `**=` are shorthand for "take the current value, apply the operation, and store the result back." The modulus operator `%` returns the remainder of a division — here 10 divided by 3 leaves a remainder of 1.

Example 2 — Comparison & logical operators

```
const isLoggedIn = true;
const role = "admin";

if (isLoggedIn && role === "admin") {
  console.log("Access granted");
}
console.log(5 > 3 || 2 > 10); // true
```

Explanation: `&&` (AND) requires both sides to be true — here both `isLoggedIn` and the role check must pass before the message logs. `||` (OR) only needs one side to be true — even though `2 > 10` is false, the whole expression is true because `5 > 3` is true.

Example 3 — Ternary operator

```
const age = 17;
const status = age >= 18 ? "Adult" : "Minor";
console.log(status); // "Minor"
```

Explanation: The ternary operator is a compact if/else in one line: `condition ? valueIfTrue : valueIfFalse` . Since 17 is not `>= 18`, the expression evaluates to the second value, "Minor".

Example 4 — Nullish coalescing (??) — safe defaults for API data

```
const req = { body: { username: null } };
const username = req.body.username ?? "Guest";
console.log(username); // "Guest" — only falls back on null/undefined, not 0 or ""
```

Explanation: `??` returns the right-hand value only when the left-hand value is exactly `null` or `undefined` . Since `username` is null, it falls back to "Guest" — but unlike `||` , it would NOT override valid falsy values like `0` or an empty string, which is safer for real form data.

Example 5 — Optional chaining (?.) — used constantly with Mongoose docs

```
const user = { profile: { address: null } };
console.log(user.profile?.address?.city); // undefined (no crash!)
console.log(user.profile?.phone ?? "Not provided");
```

Explanation: `?.` safely accesses a nested property — if any part in the chain is `null` or `undefined` , the whole expression short-circuits to `undefined` instead of throwing an error. This prevents the classic "Cannot read property of undefined" crash when API/DB data is incomplete.

4 Control Statements & Loops

Definition: Control statements (if/else, switch) decide which block of code runs based on a condition. Loops (for, while) repeat a block of code multiple times.

Real-life example: A security guard checking IDs is like an if/else — "if age >= 18, let them in, else refuse." A washing machine cycle is like a loop — it repeats the same wash-rinse-spin steps a set number of times.

Example 1 — if / else if / else

```
function checkStock(qty) {
  if (qty === 0) {
    return "Out of stock";
  } else if (qty < 5) {
    return "Low stock";
  } else {
    return "In stock";
  }
}
```

Explanation: JavaScript checks each condition top to bottom. If `qty` is exactly 0 it returns "Out of stock"; otherwise it checks the next condition, and if none match, the final `else` block runs as the fallback.

Example 2 — switch statement

```
function getStatusMessage(code) {
  switch (code) {
    case 200: return "OK";
    case 404: return "Not Found";
    case 500: return "Server Error";
    default: return "Unknown status";
  }
}
```

Explanation: `switch` compares `code` against each `case` value; when a match is found it runs that branch. If nothing matches, the `default` branch runs — this is a cleaner alternative to a long if/else-if chain when checking one value against many possibilities.

Example 3 — for and while loops

```
for (let i = 0; i < 5; i++) {
  console.log(`Page ${i}`);
}

let retries = 3;
while (retries > 0) {
  console.log(`Retrying... ${retries} left`);
  retries--;
}
```

Explanation: The `for` loop runs a set number of times (5, from i=0 to i=4) — ideal when you know the count in advance. The `while` loop keeps running as long as its condition is true; here it counts down until `retries` reaches 0, useful when the number of repetitions isn't known upfront (like network retry attempts).

Example 4 — for...of (arrays) vs for...in (object keys)

```
const users = ["Ali", "Sara", "Zain"];
for (const u of users) console.log(u);

const config = { host: "localhost", port: 5000 };
for (const key in config) {
  console.log(`${key}: ${config[key]}`);
}
```

Explanation: `for...of` loops directly over the values of an iterable like an array ("Ali", "Sara", "Zain"). `for...in` loops over the property names (keys) of an object, letting you access both the key and its value with `config[key]`.

Example 5 — break, continue & guard clauses

```
for (let i = 0; i < 10; i++) {
  if (i === 3) continue; // skip 3
  if (i === 6) break;    // stop at 6
  console.log(i);
}
```

Explanation: `continue` skips the rest of the current iteration and jumps to the next one (so 3 is never printed), while `break` exits the loop entirely (so nothing after 6 runs, including 6 itself). The output is: 0, 1, 2, 4, 5.

5 Modern JS (ES6+): let/const, Arrow Functions, Template Literals

Definition: ES6 (ECMAScript 2015) introduced a modern syntax for JavaScript, including arrow functions (a shorter way to write functions) and template literals (embedding variables directly inside strings using backticks).

Real-life example: Template literals are like a mail-merge letter template — "Dear \${customerName}, your order #\${orderId} has shipped" automatically fills in the blanks for each customer instead of you retyping the whole letter.

Example 1 — Arrow function basics

```
const add = (a, b) => a + b;
const square = n => n * n;
const greet = () => console.log("Hello!");
console.log(add(2, 3)); // 5
```

Explanation: Arrow functions drop the `function` keyword and use `=>` instead. When the body is a single expression (like `a + b`), it's automatically returned with no need for curly braces or the `return` keyword.

Example 2 — Arrow functions inside array methods (very common)

```
const nums = [1, 2, 3, 4];
const doubled = nums.map(n => n * 2);
console.log(doubled); // [2, 4, 6, 8]
```

Explanation: `map()` runs the arrow function `n => n * 2` on every item in the array, collecting the results into a brand-new array — the original `nums` array is left unchanged.

Example 3 — Template literals for strings

```
const name = "Bilal";
const cartTotal = 4500;
console.log(`Hi ${name}, your total is Rs. ${cartTotal}.`);

// Multi-line strings without concatenation:
const emailBody = `Dear ${name},
Your order has been shipped.
Thank you for shopping with us.`;
```

Explanation: Backticks (```) let you embed variables directly inside `${ }` placeholders instead of using `+` concatenation. They also allow strings to span multiple lines naturally, which is perfect for building emails or messages.

Example 4 — Arrow functions and `this` (no re-binding, unlike regular functions)

```
class Timer {
  constructor() {
    this.seconds = 0;
  }
  start() {
    setInterval(() => {
      this.seconds++; // arrow keeps "this" = Timer instance
      console.log(this.seconds);
    }, 1000);
  }
}
```

Explanation: A normal function would create its own separate `this` when called inside `setInterval`, losing access to the Timer instance. An arrow function does not have its own `this` — it borrows `this` from where it was defined (inside the `start()` method), so `this.seconds` correctly refers to the Timer's own property.

Example 5 — Default parameters

```
function createUser(name, role = "user", isActive = true) {
  return { name, role, isActive };
}
console.log(createUser("Hina"));
// { name: "Hina", role: "user", isActive: true }
```

Explanation: If a caller doesn't provide a value for `role` or `isActive`, JavaScript automatically uses the default values given after the `=` sign — avoiding manual "if undefined, set default" checks inside the function body.

6 Destructuring, Spread & Rest Operators

Definition: Destructuring unpacks values from arrays or objects into separate variables. The spread operator (`...`) expands an array/object into individual items. The rest operator (also `...`) does the opposite — it gathers multiple items into a single array.

Real-life example: Destructuring is like unpacking a grocery bag straight into labeled kitchen containers instead of leaving everything in one bag. Spread is like pouring one jar's contents into a bigger jar along with other ingredients; rest is like sweeping up all the leftover crumbs into one pile.

Example 1 — Object destructuring (extremely common in Express)

```
app.post("/api/users", (req, res) => {
  const { name, email, password } = req.body;
  console.log(name, email);
});
```

Explanation: Instead of writing `req.body.name` , `req.body.email` , and so on separately, destructuring pulls out `name` , `email` , and `password` as standalone variables in one line, matching the key names in the incoming JSON body.

Example 2 — Array destructuring

```
const coordinates = [31.5, 74.3];
const [lat, lng] = coordinates;

// Classic React Hook pattern uses array destructuring:
// const [count, setCount] = useState(0);
```

Explanation: Array destructuring assigns items by position: the first element goes into `lat` , the second into `lng` . React's `useState` Hook returns a two-item array (current value and setter function), which is why it's always destructured the same way.

Example 3 — Spread operator (copy & merge)

```
const defaults = { theme: "light", notifications: true };
const userPrefs = { theme: "dark" };
const finalSettings = { ...defaults, ...userPrefs };
console.log(finalSettings); // { theme: "dark", notifications: true }

const arr1 = [1, 2];
const arr2 = [...arr1, 3, 4]; // [1, 2, 3, 4]
```

Explanation: Spreading `...defaults` then `...userPrefs` into a new object copies all properties from both — since `userPrefs` comes second, its `theme: "dark"` overwrites the default. The same `...` syntax on arrays copies existing items into a new array while adding more.

Example 4 — Rest operator (collect remaining values)

```
function sum(...numbers) {
  return numbers.reduce((acc, n) => acc + n, 0);
}
console.log(sum(1, 2, 3, 4)); // 10

const { id, ...otherFields } = { id: 1, name: "A", price: 99 };
console.log(otherFields); // { name: "A", price: 99 }
```

Explanation: In the function, `...numbers` gathers every argument passed in, no matter how many, into a single array. In the destructuring example, `...otherFields` collects every property except `id` (which was pulled out separately) into its own object.

Example 5 — Immutable state updates in React (spread pattern)

```
const [todos, setTodos] = useState([]);

const addTodo = (newTodo) => {
  setTodos(prev => [...prev, newTodo]); // never mutate state directly
};
```

Explanation: React requires state to be replaced, not mutated in place. `[...prev, newTodo]` creates a brand-new array containing all previous todos plus the new one, which React can detect as a change and use to trigger a re-render.

7 ES Modules (import / export)

Definition: A module is a separate file containing reusable code. The `export` keyword makes values/functions/components available to other files, and `import` brings them in where they are needed.

Real-life example: Modules are like separate toolboxes in a workshop — the "electrical" toolbox exports specific tools, and any worker who needs them simply imports that toolbox instead of building duplicate tools from scratch.

Example 1 — Named exports

```
// utils/math.js
export const add = (a, b) => a + b;
export const subtract = (a, b) => a - b;
```

Explanation: Adding `export` before a declaration makes that specific value available to other files. A file can have multiple named exports, each imported individually by name elsewhere.

Example 2 — Importing named exports

```
// app.js
import { add, subtract } from "./utils/math.js";
console.log(add(2, 3));
```

Explanation: The curly braces around `{ add, subtract }` must match the exact names used in the export statement, and the path (`"./utils/math.js"`) tells JavaScript where to find the file relative to the current one.

Example 3 — Default export (common for React components)

```
// components/Navbar.jsx
export default function Navbar() {
  return <nav>My App</nav>;
}
```

```
// App.jsx
import Navbar from "./components/Navbar";
```

Explanation: A file can have only one `default` export, representing its "main" piece of code — here, the Navbar component. When importing a default export, no curly braces are needed and you can name it whatever you like on import.

Example 4 — Node.js Express route module

```
// routes/userRoutes.js
import express from "express";
const router = express.Router();

router.get("/", (req, res) => res.send("All users"));
export default router;
```

```
// server.js
import userRoutes from "./routes/userRoutes.js";
app.use("/api/users", userRoutes);
```

Explanation: The routes file builds a mini-router and exports it as the default export. The main server file imports that router and "mounts" it at the `/api/users` path — this is how large Express apps stay organized across many route files instead of one giant server.js.

Example 5 — Mixing default and named exports

```
// models/Product.js
import mongoose from "mongoose";
export const COLLECTION_NAME = "products";
const Product = mongoose.model("Product", new mongoose.Schema({ name: String }));
export default Product;
```

Explanation: A single file can export both a default value (the `Product` model, most files' "main" purpose) and additional named exports (`COLLECTION_NAME`) for smaller supporting values, importable separately as needed.

8 Arrays & Array Methods

Definition: An array is an ordered list of values stored in a single variable. Array methods like `map`, `filter`, and `reduce` transform, filter, or combine those values without manually writing loops.

Real-life example: An array is like a numbered shelf of items. "map" is like relabeling every item on the shelf with a new price tag; "filter" is like pulling out only the items on sale; "reduce" is like adding up the total cost of everything on the shelf.

Example 1 — map (transform every item, used to render JSX lists)

```
const products = [{ name: "Pen", price: 10 }, { name: "Book", price: 50 }];
const names = products.map(p => p.name);
console.log(names); // ["Pen", "Book"]
```

Explanation: `map()` visits every product object and pulls out just its `name`, building a brand-new array of names. This exact pattern is how React turns an array of data into a list of JSX elements on screen.

Example 2 — filter (keep matching items)

```
const users = [{ name: "Ali", active: true }, { name: "Sam", active: false }];
const activeUsers = users.filter(u => u.active);
console.log(activeUsers); // [{ name: "Ali", active: true }]
```

Explanation: `filter()` tests every item against the condition `u.active`; only items where that test returns true make it into the new array — Sam is excluded because his `active` field is false.

Example 3 — reduce (aggregate values — e.g. cart total)

```
const cart = [{ price: 100 }, { price: 250 }, { price: 50 }];
const total = cart.reduce((sum, item) => sum + item.price, 0);
console.log(total); // 400
```

Explanation: `reduce()` walks through the array while carrying an accumulator (`sum`), starting at 0. On each step it adds the current item's price to the running total, ending with the combined sum of all three prices.

Example 4 — find, some, every

```
const nums = [4, 8, 15, 16];
console.log(nums.find(n => n > 10)); // 15 (first match)
console.log(nums.some(n => n > 10)); // true
console.log(nums.every(n => n > 0)); // true
```

Explanation: `find()` returns the first item that matches the condition (15, the first value over 10). `some()` checks if at least one item matches (true, since 15 and 16 qualify). `every()` checks if ALL items match a condition (true, since every number is greater than 0).

Example 5 — sort, includes, and chaining methods

```
const scores = [55, 10, 90, 33];
const sorted = [...scores].sort((a, b) => a - b);
console.log(sorted); // [10, 33, 55, 90]

// Chaining: filter then map — common when shaping API responses
const passed = scores.filter(s => s >= 50).map(s => `${s}%`);
console.log(passed); // ["55%", "90%"]
```

Explanation: Spreading into a new array before `sort()` avoids mutating the original `scores` array. `(a, b) => a - b` sorts numbers in ascending order. The last line chains `filter()` (keep scores `>= 50`) directly into `map()` (format them as percentage strings) in a single readable pipeline.

9 Strings & String Methods

Definition: A string is a sequence of characters representing text. String methods let you search, transform, split, or clean up text values.

Real-life example: Editing a string is like proofreading a document — trimming extra spaces, converting SHOUTING text to normal case, or splitting a long sentence into separate words — all without retyping anything by hand.

Example 1 — Basic slicing & case methods

```
const email = "Test@Example.com";
console.log(email.toLowerCase()); // "test@example.com"
console.log(email.slice(0, 4)); // "Test"
```

Explanation: `toLowerCase()` converts every letter to lowercase (useful for case-insensitive email comparisons before saving to a database). `slice(0, 4)` extracts characters from index 0 up to (but not including) index 4, giving the first four characters.

Example 2 — trim & validation (common before saving to DB)

```
const input = "  hello world  ";
console.log(input.trim()); // "hello world"
console.log(input.trim().length); // 11
```

Explanation: `trim()` removes whitespace from both ends of a string — commonly used to clean up user input from a form before validating or saving it. `.length` then tells you how many characters remain.

Example 3 — includes, startsWith, replace

```
const filename = "profile-picture.png";
console.log(filename.includes(".png")); // true
console.log(filename.startsWith("profile")); // true
console.log(filename.replace(".png", ".jpg"));
```

Explanation: `includes()` checks whether a substring exists anywhere inside the string (true, since ".png" is present). `startsWith()` checks the beginning specifically. `replace()` swaps the first matching occurrence of one substring with another, producing "profile-picture.jpg".

Example 4 — split & join (turning strings into arrays and back)

```
const tags = "react,node,mongodb";
const tagArray = tags.split(",");
console.log(tagArray); // ["react", "node", "mongodb"]
console.log(tagArray.join(" | ")); // "react | node | mongodb"
```

Explanation: `split(",")` breaks the string into an array wherever a comma appears. `join(" | ")` does the reverse — it glues the array back into a single string, inserting " | " between each item.

Example 5 — Template-based slug generation

```
function toSlug(title) {
  return title
    .toLowerCase()
    .trim()
    .replace(/\s+/g, "-");
}
console.log(toSlug("  Best MERN Stack Course  "));
// "best-mern-stack-course"
```

Explanation: This chains three string methods together: first lowercase the title, then trim outer whitespace, then replace every run of one-or-more spaces (`/\s+/g`) with a single dash — producing a clean, URL-friendly "slug" commonly used for blog post or product URLs.

10 Date and Time

Definition: The `Date` object represents a specific point in time, letting you create, compare, and format dates and timestamps.

Real-life example: Think of a hotel check-in system — it records the exact check-in timestamp, calculates the checkout date by adding a number of days, and displays it in a human-friendly format on the receipt.

Example 1 — Creating dates

```
const now = new Date();
const specific = new Date("2026-01-15");
console.log(now.toISOString()); // e.g. "2026-07-16T10:00:00.000Z"
```

Explanation: `new Date()` with no arguments captures the current moment. Passing a date string creates a `Date` object for that specific day. `toISOString()` formats a `Date` into the standardized ISO format, which is exactly how MongoDB stores date fields internally.

Example 2 — Getting date parts

```
const date = new Date();
console.log(date.getFullYear()); // 2026
console.log(date.getMonth());    // 0-indexed (0 = January)
console.log(date.getDate());     // day of month
console.log(date.getDay());      // day of week (0 = Sunday)
```

Explanation: Each getter method pulls out a single piece of the date. A common gotcha: `getMonth()` counts from 0, so January is 0 and December is 11 — always remember to add 1 when displaying the month to a user.

Example 3 — Timestamps for comparing/expiry (e.g. JWT or session logic)

```
const createdAt = new Date();
const expiresAt = new Date(createdAt.getTime() + 15 * 60 * 1000); // +15 min
console.log(Date.now() > expiresAt.getTime()); // true if expired
```

Explanation: `getTime()` converts a date into a raw millisecond number, which makes arithmetic easy — adding 15 minutes ($15 \times 60 \text{ seconds} \times 1000 \text{ milliseconds}$) gives an expiry timestamp. Comparing `Date.now()` to that expiry number is exactly how session or password-reset-link expiry is implemented.

Example 4 — Formatting dates for the UI

```
const post = { publishedAt: new Date("2026-03-10") };
console.log(post.publishedAt.toLocaleDateString("en-US", {
  year: "numeric", month: "long", day: "numeric"
})); // "March 10, 2026"
```

Explanation: `toLocaleDateString()` converts a raw `Date` object into a human-readable format based on locale rules and the options provided — here producing a friendly "March 10, 2026" instead of a raw ISO timestamp for display in a React UI.

Example 5 — Mongoose automatically stores Date objects

```
// Mongoose schema (backend) - timestamps: true adds createdAt & updatedAt
const orderSchema = new mongoose.Schema(
  { product: String, qty: Number },
  { timestamps: true }
);
// Every saved document automatically gets createdAt / updatedAt Date fields
```

Explanation: Passing `{ timestamps: true }` as a schema option tells Mongoose to automatically add and maintain `createdAt` and `updatedAt` `Date` fields on every document, without you writing any manual date-setting code.

11 Math Object

Definition: `Math` is a built-in JavaScript object providing constants and functions for numeric operations like rounding, finding min/max, and generating random numbers.

Real-life example: A cashier rounding your bill to the nearest rupee uses rounding logic just like `Math.round()` ; a lottery machine picking a random number is just like `Math.random()` .

Example 1 — Rounding methods

```
console.log(Math.round(4.5)); // 5
console.log(Math.floor(4.9)); // 4
console.log(Math.ceil(4.1)); // 5
```

Explanation: `round()` rounds to the nearest whole number, `floor()` always rounds down, and `ceil()` always rounds up — regardless of how close the decimal is to the next whole number.

Example 2 — Min, max & power

```
console.log(Math.max(10, 25, 3)); // 25
console.log(Math.min(10, 25, 3)); // 3
console.log(Math.pow(2, 3)); // 8
```

Explanation: `Math.max()` and `Math.min()` accept any number of arguments and return the largest/smallest. `Math.pow(2, 3)` raises 2 to the power of 3 ($2 \times 2 \times 2 = 8$).

Example 3 — Generating a random OTP

```
function generateOTP() {
  return Math.floor(100000 + Math.random() * 900000);
}
console.log(generateOTP()); // e.g. 483920 (6-digit OTP)
```

Explanation: `Math.random()` generates a decimal between 0 and 1. Multiplying by 900000 and adding 100000 shifts that range so the result always falls between 100000 and 999999. `Math.floor()` then strips off the decimal, leaving a clean 6-digit whole number — perfect for a one-time password.

Example 4 — Calculating pagination (total pages)

```
const totalItems = 95;
const pageSize = 10;
const totalPages = Math.ceil(totalItems / pageSize);
console.log(totalPages); // 10
```

Explanation: Dividing 95 items by a page size of 10 gives 9.5. `Math.ceil()` rounds that up to 10, ensuring the leftover 5 items still get their own final page — a standard pagination calculation used on any product listing or admin table.

Example 5 — Formatting price / discount calculations

```
function applyDiscount(price, percent) {
  const discounted = price - (price * percent) / 100;
  return Math.round(discounted * 100) / 100; // round to 2 decimals
}
console.log(applyDiscount(499.99, 15)); // 424.99
```

Explanation: The function first subtracts the discount amount from the price. Because currency values shouldn't show more than 2 decimal places, multiplying by 100, rounding to the nearest whole number, then dividing by 100 cleanly trims the result to exactly 2 decimal places.

12 DOM, BOM & Window Object

Definition: The DOM (Document Object Model) is a tree-like representation of an HTML page that JavaScript can read and modify. The BOM (Browser Object Model) exposes browser-level features like the URL, screen size, and storage, accessed through the global `window` object.

Real-life example: The DOM is like the furniture layout of a house that you can rearrange (move a chair, repaint a wall); the BOM is like the house's utilities — the front door (navigation), the mailbox (localStorage), and the windows for looking outside (screen info).

Example 1 — Selecting and updating elements

```
const heading = document.querySelector("h1");
heading.textContent = "Welcome back!";
heading.classList.add("highlight");
```

Explanation: `querySelector("h1")` finds the first `<h1>` element on the page. Setting `textContent` changes its visible text, and `classList.add()` attaches a CSS class to it — a common way to visually style an element after some event.

Example 2 — Creating and appending elements

```
const list = document.getElementById("todo-list");
const item = document.createElement("li");
item.textContent = "Buy groceries";
list.appendChild(item);
```

Explanation: This builds a new `<li>` element from scratch in memory, fills it with text, then attaches it as a child of the existing `#todo-list` element — making it appear on the page.

Example 3 — window / BOM properties

```
console.log(window.innerWidth, window.innerHeight);
console.log(window.location.href); // current page URL
window.location.assign("/dashboard"); // navigate programmatically
```

Explanation: `window.innerWidth/innerHeight` report the visible browser viewport size. `window.location.href` gives the current page's full URL, and calling `location.assign()` redirects the browser to a new page — useful after a successful login, for example.

Example 4 — localStorage (client-side persistence, e.g. saving a JWT)

```
localStorage.setItem("token", "jwt_abc123");
const token = localStorage.getItem("token");
localStorage.removeItem("token"); // on logout
```

Explanation: `localStorage` is part of the BOM and persists key/value data in the browser even after the page is refreshed. Here a login token is saved, later retrieved to authenticate API requests, and removed on logout to clear the session.

Example 5 — Why React abstracts direct DOM manipulation

```
// Vanilla JS: manually find & update the DOM
document.getElementById("count").textContent = count;

// React: describe the UI, React updates the real DOM for you
function Counter() {
  const [count, setCount] = useState(0);
  return <p>{count}</p>; // React re-renders automatically on state change
}
```

Explanation: The vanilla JS version requires you to manually find the element and update it every time the value changes. The React version simply declares what the UI should look like based on `count`; React handles finding and updating the actual DOM element behind the scenes whenever `setCount` is called.

13 Events in JavaScript

Definition: An event is an action that happens in the browser — a click, a keypress, a form submission — that JavaScript can "listen" for and respond to with a function.

Real-life example: A doorbell is an event system — pressing the button (the event) triggers a chime (the event handler function) inside the house.

Example 1 — Vanilla JS click event

```
const btn = document.querySelector("#saveBtn");
btn.addEventListener("click", () => {
  console.log("Save clicked!");
});
```

Explanation: `addEventListener("click", ...)` attaches a function that runs every time the button is clicked. The function itself (an arrow function here) is called the event handler.

Example 2 — Form submit event & preventDefault

```
const form = document.querySelector("#loginForm");
form.addEventListener("submit", (e) => {
  e.preventDefault(); // stop full-page reload
  console.log("Form submitted via JS");
});
```

Explanation: By default, submitting an HTML form reloads the entire page. `e.preventDefault()` stops that default browser behavior so JavaScript can handle the submission instead — typically by sending the data to an API with `fetch()` without a page refresh.

Example 3 — React's synthetic event handling

```
function LoginForm() {
  const handleSubmit = (e) => {
    e.preventDefault();
    console.log("Logging in...");
  };
  return (
    <form onSubmit={handleSubmit}>
      <button type="submit">Login</button>
    </form>
  );
}
```

Explanation: React lets you attach event handlers directly as JSX props (`onSubmit={handleSubmit}`) instead of calling `addEventListener` manually. React still uses `e.preventDefault()` the same way underneath to stop the default page reload.

Example 4 — Event delegation (attach one listener for many children)

```
document.querySelector("#todo-list").addEventListener("click", (e) => {
  if (e.target.matches(".delete-btn")) {
    e.target.closest("li").remove();
  }
});
```

Explanation: Instead of adding a separate click listener to every delete button (which could be dozens), one listener is placed on the parent list. `e.target` tells us exactly which child element was actually clicked, and if it matches a delete button, its closest parent `<li>` is removed. This is more efficient, especially for lists that grow dynamically.

Example 5 — Input change event (controlled input in React)

```
function SearchBox() {
  const [query, setQuery] = useState("");
  return (
    <input
      value={query}
      onChange={(e) => setQuery(e.target.value)}
    />
  );
}
```

Explanation: Every keystroke fires the `onChange` event, reading the input's current text via `e.target.value` and storing it in state with `setQuery`. Because the input's `value` is tied back to that state, React fully controls what's displayed — this is called a "controlled input."

14 Timing Events: `setTimeout` & `setInterval`

Definition: `setTimeout()` runs a function once after a specified delay. `setInterval()` repeatedly runs a function every fixed interval until stopped.

Real-life example: A microwave timer is like `setTimeout` — it does one thing (beeps) once, after a delay. A heartbeat monitor beeping every second is like `setInterval` — it repeats forever until switched off.

Example 1 — `setTimeout` basics

```
setTimeout(() => {  
  console.log("Ran after 2 seconds");  
}, 2000);
```

Explanation: The function passed to `setTimeout` is scheduled to run once after the given delay in milliseconds (2000 ms = 2 seconds) — the rest of the program keeps running normally while it waits.

Example 2 — `setInterval` & `clearInterval`

```
let count = 0;  
const intervalId = setInterval(() => {  
  count++;  
  console.log(`Tick ${count}`);  
  if (count === 5) clearInterval(intervalId); // stop after 5 ticks  
, 1000);
```

Explanation: `setInterval` repeats its function every 1000ms (1 second) and returns an ID we save as `intervalId`. Once `count` reaches 5, `clearInterval(intervalId)` stops the repetition — without this, it would tick forever.

Example 3 — Debouncing a search input (very common React pattern)

```
function useDebouncedSearch(callback, delay) {  
  let timer;  
  return (query) => {  
    clearTimeout(timer);  
    timer = setTimeout(() => callback(query), delay);  
  };  
}  
const searchAPI = useDebouncedSearch(q => console.log("Searching:", q), 400);
```

Explanation: Every time the returned function is called (e.g. on each keystroke), it first cancels any pending timer with `clearTimeout`, then starts a fresh one. The actual search only fires once the user stops typing for 400ms straight — preventing an API call on every single keystroke.

Example 4 — Cleanup timers inside a React `useEffect`

```
useEffect(() => {  
  const id = setInterval(() => console.log("Polling server..."), 5000);  
  return () => clearInterval(id); // cleanup on unmount  
, []);
```

Explanation: The function returned from inside `useEffect` is a cleanup function that React automatically runs when the component is removed from the screen. Clearing the interval here prevents the polling from continuing to run in the background after the user navigates away, which would otherwise cause a memory leak.

Example 5 — Simulated retry logic in a Node.js service

```
function retryConnection(attempt = 1) {  
  console.log(`Attempt ${attempt} to connect to DB...`);  
  setTimeout(() => {  
    if (attempt < 3) retryConnection(attempt + 1);  
  }, 1000 * attempt);  
}
```

Explanation: This function calls itself again after a delay that grows with each attempt (1s, then 2s, then 3s) — a simple version of "exponential backoff," commonly used when retrying a failed database or API connection instead of hammering it instantly and repeatedly.

15 Scope, Hoisting & Closures

Definition: Scope determines where a variable can be accessed. Hoisting is JavaScript's behavior of moving variable/function declarations to the top of their scope before code runs. A closure is a function that "remembers" the variables from the scope it was created in, even after that outer scope has finished running.

Real-life example: A closure is like a backpack a function carries with it — even after leaving the room (scope) where it was packed, the function still has access to whatever was inside that backpack.

Example 1 — Function scope vs block scope

```
function demo() {
  if (true) {
    var a = 1;    // function-scoped, leaks outside the if-block
    let b = 2;    // block-scoped, stays inside
  }
  console.log(a); // 1
  // console.log(b); ✖ ReferenceError
}
```

Explanation: `var` ignores block boundaries and is only limited by the surrounding function — so `a` is still readable outside the `if` block. `let` respects block boundaries, so `b` only exists inside the `if` block and would throw an error if accessed outside it.

Example 2 — Hoisting behavior

```
console.log(x); // undefined (var is hoisted, initialized as undefined)
var x = 5;

console.log(y); // ✖ ReferenceError (let is hoisted but NOT initialized - "temporal dead zone")
let y = 10;
```

Explanation: JavaScript moves declarations to the top of their scope before running the code. `var x` is hoisted and pre-filled with `undefined`, so reading it early gives undefined rather than an error. `let y` is also hoisted, but stays in an inaccessible "temporal dead zone" until its actual declaration line runs — accessing it early throws an error instead.

Example 3 — Basic closure

```
function makeCounter() {
  let count = 0;
  return function() {
    count++;
    return count;
  };
}
const counter = makeCounter();
console.log(counter()); // 1
console.log(counter()); // 2 - "count" is remembered between calls
```

Explanation: `makeCounter()` runs once and returns an inner function. Even though `makeCounter()` has already finished executing, the returned function still remembers and can update the `count` variable from its original scope — that "memory" is exactly what a closure is.

Example 4 — Closures for private data (module pattern)

```
function createBankAccount(balance) {
  return {
    deposit: (amt) => (balance += amt),
    getBalance: () => balance
  };
}
const account = createBankAccount(1000);
account.deposit(500);
console.log(account.getBalance()); // 1500 (balance is private, no direct access)
```

Explanation: `balance` is only accessible through the `deposit` and `getBalance` functions returned by `createBankAccount` — there's no way to reach into `account` and change `balance` directly from outside, giving it true privacy.

Example 5 — Closures inside React custom Hooks

```
function useCounter(initial = 0) {
  const [count, setCount] = useState(initial);
  const increment = () => setCount(c => c + 1); // closure over setCount
  return { count, increment };
}
```

Explanation: The `increment` function is a closure that "remembers" the `setCount` function from this Hook call, even when it's used later inside a button's `onClick` deep in a component tree.

16 Asynchronous JS: Callbacks, Promises, Async/Await

Definition: Asynchronous code lets JavaScript start a slow operation (like a database query) and continue running other code while it waits, instead of freezing everything. Callbacks, Promises, and async/await are three progressively cleaner ways to handle that waiting.

Real-life example: Ordering food at a restaurant is asynchronous — you place your order (start the operation), the kitchen works on it in the background, and you're notified (callback/promise resolves) when it's ready, instead of standing frozen at the counter until it's done.

Example 1 — Callback-based async (older Node.js style)

```
const fs = require("fs");
fs.readFile("data.txt", "utf8", (err, data) => {
  if (err) return console.error(err);
  console.log(data);
});
```

Explanation: `readFile` starts reading the file and immediately returns control to the rest of the program. Once the file is fully read (which can take time), Node calls the callback function with either an error or the file's data — this "call me back later" pattern is where the name "callback" comes from.

Example 2 — Creating and using a Promise

```
function fetchUser(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      id > 0 ? resolve({ id, name: "Ayesha" }) : reject("Invalid ID");
    }, 500);
  });
}
fetchUser(1).then(user => console.log(user)).catch(err => console.error(err));
```

Explanation: A Promise represents a value that will be available later. Inside it, calling `resolve()` marks it as successful, while `reject()` marks it as failed. `.then()` runs when the promise resolves successfully, and `.catch()` runs if it was rejected.

Example 3 — async/await with try/catch (standard Express controller style)

```
app.get("/api/products/:id", async (req, res) => {
  try {
    const product = await Product.findById(req.params.id);
    if (!product) return res.status(404).json({ message: "Not found" });
    res.json(product);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});
```

Explanation: The `async` keyword marks this route handler as asynchronous, allowing the use of `await` inside it. `await Product.findById(...)` pauses just this function until the database returns a result, without blocking other incoming requests. `try/catch` then catches any database errors and responds with a proper error status instead of crashing the server.

Example 4 — Promise.all (parallel requests, e.g. React data fetching)

```
async function loadDashboard() {
  const [users, orders] = await Promise.all([
    fetch("/api/users").then(r => r.json()),
    fetch("/api/orders").then(r => r.json())
  ]);
  console.log(users, orders);
}
```

Explanation: `Promise.all()` runs both fetch requests at the same time rather than one after another, and waits for both to finish before continuing. This is much faster than awaiting each request separately when they don't depend on each other.

Example 5 — Fetching data inside a React useEffect

```
useEffect(() => {
  async function loadPosts() {
    const res = await fetch("/api/posts");
    const data = await res.json();
    setPosts(data);
  }
  loadPosts();
}, []);
```

Explanation: An inner `async` function is defined and immediately called inside `useEffect` (since the effect callback itself can't directly be async). It fetches posts, waits for the JSON body to be parsed, then stores the result in state so the component re-renders with the new data. The empty `[]` dependency array means this runs once when the component first mounts.

17 The Event Loop

Definition: The event loop is the mechanism that lets JavaScript's single thread handle asynchronous tasks — it continuously checks whether the call stack is empty and, if so, pulls the next waiting task (from the microtask or macrotask queue) to execute.

Real-life example: A single waiter serving multiple restaurant tables is like the event loop — instead of standing at one table until that customer finishes their entire meal, the waiter takes an order, moves to the next table while the kitchen prepares the first order, and comes back when it's ready.

Example 1 — Call stack vs callback queue ordering

```
console.log("1: start");
setTimeout(() => console.log("2: timeout"), 0);
console.log("3: end");
// Output: 1: start, 3: end, 2: timeout
// setTimeout always runs AFTER the synchronous code finishes
```

Explanation: Even with a 0ms delay, `setTimeout`'s callback is placed in a queue and can only run once all currently running (synchronous) code has finished. That's why "3: end" logs before "2: timeout", even though the timeout was scheduled earlier in the code.

Example 2 — Microtasks (Promises) run before macrotasks (setTimeout)

```
console.log("A");
setTimeout(() => console.log("B (macrotask)"), 0);
Promise.resolve().then(() => console.log("C (microtask)"));
console.log("D");
// Output: A, D, C, B
```

Explanation: Promise callbacks (microtasks) are always processed before `setTimeout` callbacks (macrotasks), even if both are scheduled to run "immediately." So after the synchronous "A" and "D" print, the queued Promise callback "C" runs before the queued `setTimeout` "B".

Example 3 — Why this matters for Express (non-blocking I/O)

```
app.get("/slow", async (req, res) => {
  const data = await SomeModel.find(); // DB call doesn't block other requests
  res.json(data);
});
// While this request awaits the DB, Node's event loop keeps
// serving other incoming requests on the same thread.
```

Explanation: While this particular request is waiting on the database, Node doesn't sit idle — the event loop is free to pick up and process other incoming requests, which is why a single Node.js server thread can serve many users concurrently despite being single-threaded.

Example 4 — Blocking the event loop (anti-pattern to avoid)

```
// ✗ BAD: synchronous heavy loop blocks ALL requests
function blockingTask() {
  let total = 0;
  for (let i = 0; i < 1e10; i++) total += i;
  return total;
}
// ✓ GOOD: offload heavy work to a worker thread or a separate service
```

Explanation: This loop runs entirely synchronously and takes a long time to finish. Because JavaScript is single-threaded, nothing else — including other users' requests — can be processed until this loop completes, freezing the whole server. Heavy computational work should be moved off the main thread (e.g. to a worker thread or a separate microservice).

Example 5 — async/await is still built on the event loop

```
async function demo() {
  console.log("1");
  await null; // yields to the event loop
  console.log("3");
}
demo();
console.log("2");
// Output: 1, 2, 3 — code after "await" always runs as a microtask
```

Explanation: The function runs synchronously up to the `await`, printing "1". At that point it pauses and hands control back to the rest of the program, which prints "2". Everything after `await` resumes as a microtask, so "3" prints last — demonstrating that `async/await` is just a cleaner syntax layered over the same underlying event loop and microtask queue.

18 Prototypes, this, call, apply & bind

Definition: A prototype is an object that another object automatically inherits properties and methods from. `this` refers to the object a function is currently "attached to" when called. `call`, `apply`, and `bind` are methods that let you explicitly control what `this` refers to inside a function.

Real-life example: Prototypes are like a family recipe passed down through generations — you don't rewrite the recipe for every child, they simply inherit it. `this` is like the word "I" in a sentence — its meaning changes depending on who is speaking (which object is calling the function).

Example 1 — Prototype-based inheritance

```
function Animal(name) {
  this.name = name;
}
Animal.prototype.speak = function() {
  console.log(`${this.name} makes a sound`);
};
const dog = new Animal("Dog");
dog.speak(); // "Dog makes a sound"
```

Explanation: Adding `speak` to `Animal.prototype` means every object created with `new Animal(...)` automatically has access to that method, without each individual object needing its own copy — saving memory and keeping shared behavior in one place.

Example 2 — ES6 classes (syntactic sugar over prototypes)

```
class User {
  constructor(name, email) {
    this.name = name;
    this.email = email;
  }
  getSummary() {
    return `${this.name} (${this.email})`;
  }
}
const u = new User("Omar", "omar@mail.com");
console.log(u.getSummary());
```

Explanation: The `class` syntax is a cleaner way to write the same prototype-based pattern from Example 1. `constructor()` runs when a new object is created, setting up its properties, and `getSummary()` becomes available on every instance through the class's prototype.

Example 3 — Understanding "this" losing context

```
const user = {
  name: "Fatima",
  greet() { console.log(`Hi, ${this.name}`); }
};
const greetFn = user.greet;
greetFn(); // "Hi, undefined" — "this" is lost when detached from the object
```

Explanation: When `greet` is called as `user.greet()`, `this` refers to `user`. But once we assign the function to a plain variable (`greetFn`) and call it alone, it's no longer attached to `user`, so `this.name` is undefined.

Example 4 — Fixing it with call, apply & bind

```
function introduce(role) {
  console.log(`${this.name} works as a ${role}`);
}
const emp = { name: "Hamza" };

introduce.call(emp, "Developer"); // args listed individually
introduce.apply(emp, ["Developer"]); // args passed as an array

const boundIntroduce = introduce.bind(emp);
boundIntroduce("Team Lead"); // "this" permanently bound to emp
```

Explanation: `call()` and `apply()` both immediately run the function with `this` forced to be `emp` — they only differ in how extra arguments are passed (individually vs. as an array). `bind()` doesn't run the function right away; instead it returns a brand-new function permanently locked to use `emp` as `this`, which can be called later.

Example 5 — bind used for Express middleware / class methods

```
class AuthController {
  constructor() {
    this.secret = "jwt-secret";
    this.login = this.login.bind(this); // keep "this" correct when used as a callback
  }
  login(req, res) {
    res.json({ message: `Logged in using ${this.secret}` });
  }
}
const controller = new AuthController();
app.post("/login", controller.login); // works correctly thanks to bind
```

Explanation: When Express calls `controller.login` as a standalone callback, it would normally lose its connection to the `controller` instance (the same problem as Example 3). Binding it inside the constructor permanently attaches `this` to the instance, so `this.secret` still works correctly no matter how the function is later called.

Compiled reference notes — focused on concepts, not interview trivia

